WEEK 1 · LESSON 1 OF 24 · HANDOUT

From Chatbots to Large Language Models

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=LPZh9BOjkQs — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Intro to Large Language Models (1-hour talk) — https://www.youtube.com/watch?v=zjkBMFhNj_g

🎓 **Go deeper:** Andrej Karpathy: Deep Dive into LLMs like ChatGPT — https://www.youtube.com/watch?v=7xTGNNLPyMI

Your goal in this lesson: Explain in plain words what a large language model (LLM) is, how the field got here, and build the simplest possible language model in Python.

What is a language model?

A **language model** answers one question again and again: **"What comes next?"** Given "Murakoze ___", it gives a high probability to "cyane". Given "The capital of Rwanda is", it gives a high probability to "Kigali".

A **large** language model does exactly this, with billions of learned numbers (parameters), trained on a huge amount of text. ChatGPT, Claude, Gemini, Llama and Mistral are all LLMs. When you chat with one, it writes its answer **one small piece at a time**, each time choosing a likely next piece.

> *An LLM is a next-word predictor so good, and trained on so much text, that predicting well requires something that looks like knowledge and reasoning.*

A short history

Years	Idea	Limitation
1960s–1990s	Rule-based chatbots (ELIZA)	Only followed hand-written patterns
1990s–2000s	**n-gram** models: count which words follow which	Only look back 2–4 words
2013	Word embeddings (word2vec)	Meaning as numbers, but no sentences
2014–2017	Recurrent networks (LSTMs) for translation	Slow, forget long texts
2017	**The transformer** ("Attention Is All You Need")	—
2018–2020	BERT, GPT-2, GPT-3: pretrain on huge text	Raw models ramble, follow no instructions
2022 →	ChatGPT, Claude, Gemini: **instruction tuning + human feedback**	Still can be wrong (hallucinate)

Three ingredients made the jump possible: the **transformer**, **enormous text datasets**, and **GPUs** to train very large models. You met the transformer in Deep Learning & Neural Networks; this course shows how it becomes a useful AI system.

How this course works

- **Week 1:** how LLMs work inside — tokens, next-token prediction, sampling, a small GPT you train yourself, and how base models become assistants.
- **Week 2:** building with LLM APIs — prompts, structured output, tools, conversations, cost and reliability.
- **Week 3:** embeddings, vector search and **RAG** (letting a model answer from your own documents), plus evaluation.
- **Week 4:** agents, fine-tuning, image and speech generation, open models, and safety.

The text we learn from

To train our own models this week we need text. We generate thousands of short "market day" sentences with code, so everyone has the same data and nothing needs downloading:

```
import random

random.seed(0)
names = ["Aline", "Eric", "Grace", "Jean", "Diane", "Patrick", "Claudine", "Samuel", "Ange",
"Didier"]
places = ["Kigali", "Huye", "Musanze", "Rubavu", "Nyagatare", "Rusizi", "Muhanga"]
goods = ["tomatoes", "beans", "bananas", "avocados", "maize", "potatoes", "milk", "coffee",
"rice", "cassava"]
verbs = ["sells", "buys", "carries", "cooks", "packs", "counts"]
times = ["in the morning", "at noon", "in the evening", "on Saturday", "after the rain",
"before school"]
moods = ["happy", "tired", "busy", "proud", "calm", "hungry"]
templates = [
    "{n} {v} {g} in {p} {t}.",
    "In {p}, {n} {v} {g} {t}.",
    "{n} is {f} because the {g} are fresh.",
    "{t}, {n} goes to the market in {p}.",
    "{n} pays for the {g} with mobile money.",
    "The price of {g} in {p} is low {t}.",
    "{n} tells {m} that the {g} from {p} are the best.",
    "{n} and {m} share {g} {t}.",
]

def sentence():
    n, m = random.sample(names, 2)
    s = random.choice(templates).format(n=n, m=m, v=random.choice(verbs),
g=random.choice(goods),
                                        p=random.choice(places), t=random.choice(times),
f=random.choice(moods))
    return s[0].upper() + s[1:]

text = "\n".join(sentence() for _ in range(4000))
print(len(text), "characters")
print(text[:250])
```

The simplest language model: counting

A **bigram model** predicts the next word only from the current word, by counting how often each pair appears:

```
from collections import Counter, defaultdict

words = text.replace("\n", " \n ").split(" ")
following = defaultdict(Counter)
for current, nxt in zip(words, words[1:]):
    following[current][nxt] += 1

print("After 'pays' comes:", following["pays"].most_common(3))
print("After 'the' comes: ", following["the"].most_common(5))
```

After "pays" the model has only ever seen "for" — it is certain. After "the" there are many options — it must choose.

Let it write

```
def generate_bigram(start, n_words=15, seed=1):
    rng = random.Random(seed)
    out = [start]
    for _ in range(n_words):
        options = following[out[-1]]
        if not options:
            break
        choices, counts = zip(*options.items())
        out.append(rng.choices(choices, weights=counts)[0])   # pick in proportion to the
counts
    return " ".join(out).replace(" \n ", "\n")

print(generate_bigram("Grace"))
print(generate_bigram("In"))
```

The output uses real words in plausible pairs, but the sentences soon go wrong: "Grace share bananas from Musanze is low on Saturday." Every pair of words is fine; the sentence is not. The model only remembers **one** word. An LLM uses a transformer to look back over **thousands** of tokens, which is why its writing stays on topic.

Key terms

- **Language model** — predicts the next piece of text.
- **LLM** — a very large transformer language model trained on huge text collections.
- **Parameters** — the learned numbers inside the model.
- **n-gram / bigram model** — predicts from the last n − 1 words by counting.
- **Context** — the text the model can see when predicting.

Remember this: Every LLM is a next-token predictor; the transformer lets it use a long context, and scale makes its predictions remarkably capable.

Name: _____ Date: _____

Activity

Task 1

Print the five most common words after "in". Why are they all places?

Task 2

Generate ten sentences starting with "The". How many are grammatical?

Task 3

Build a **trigram** model that looks at the last two words. Is its writing better?

Task 4

Ask a chatbot "What comes after: The capital of Rwanda is" and compare with what your bigram model would say.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What question does a language model answer again and again?
   A. What comes next?
   B. What is the topic?
   C. Who wrote this?
   D. Is this true?
2. Which 2017 invention made modern LLMs possible?
   A. ELIZA
   B. word2vec
   C. The transformer
   D. The bigram model
3. What turned raw pretrained models into helpful assistants like ChatGPT and Claude?
   A. Removing the transformer
   B. Instruction tuning and human feedback
   C. More n-grams
   D. Bigger fonts
4. How much context does a bigram model use?
   A. The last 1,000 words
   B. The whole book
   C. None
   D. Only the current word
5. Why does the bigram model's writing wander off topic?
   A. It has no data
   B. It only remembers one word
   C. It is too large
   D. It uses a GPU
6. What are parameters?
   A. The training text
   B. The prompts users type
   C. The learned numbers inside the model
   D. The answers

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.